Beágyazott és Ambiens Rendszerek
2025. 10. 02.

Megszakítások kezelése

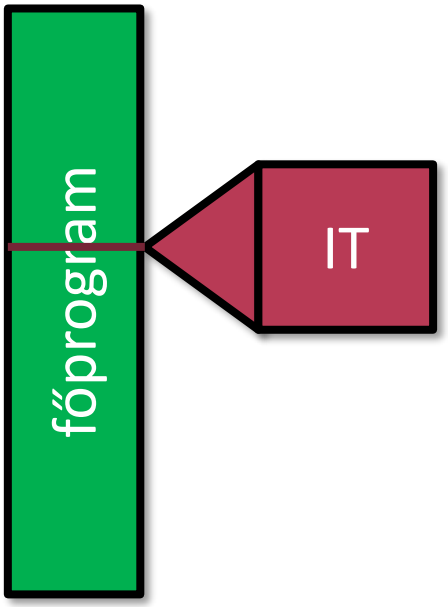


Mesterséges Intelligencia és
Rendszertervezés
Tanszék

© BME-MIT

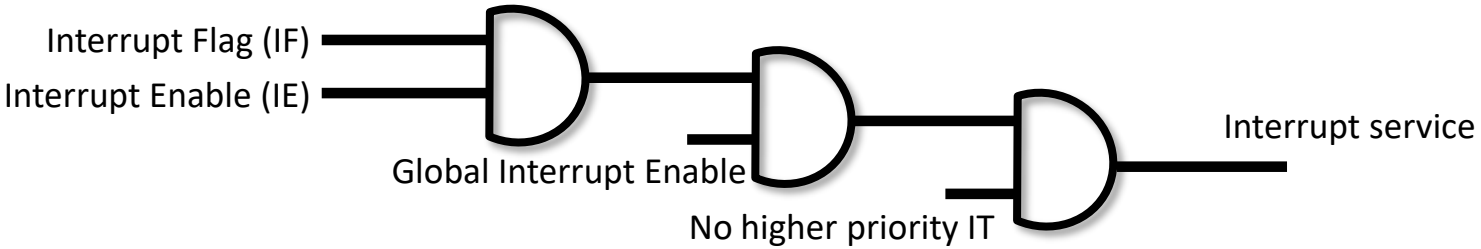
Megszakítás (interrupt)

- A processzorban külső esemény hatására megszakad a program futása, és az eseményhez tartozó kód kezd futni: beékelődik a főprogram futásába
- Fontos, hogy visszatérve a megszakításból, az eredeti futó kód „ne vegye észre”, hogy egy másik folyamat megszakította a futást. Ehhez például:
  - Vissza kell állítani a munkaregisztereket
  - Vissza kell állítani a processzor állapotregisztereit
  - Vissza kell állítani a stacket
  - Összefoglalóan: megfelelő kontextusváltás szükséges.
- Beágyazott rendszerek: rengeteg különböző architektúra és megoldás, így az általános javaslatokat eszközfüggő módon ki kell egészíteni vagy módosítani.



Megszakítás-engedélyezési hierarchia

- Általában a következő a megszakítás érvényre jutásának a folyamata:
  - IT esemény létrejötte → hozzá tartozó IT flag beáll 1-es értékre
  - Ha az eseményhez tartozó IT engedélyezve van, akkor a globális IT flag is 1-es lesz
  - Ha a globális IT engedélyezve van, akkor a megszakítás érvényre juthat
    - Prioritásos megszakításkezelés esetén magasabb prioritású IT megszakíthatja egy alacsonyabb prioritású IT rutin futását (preemptív)
    - Ha nem prioritásos az IT kezelés, akkor az IT rutinok nem szakíthatják meg egymást

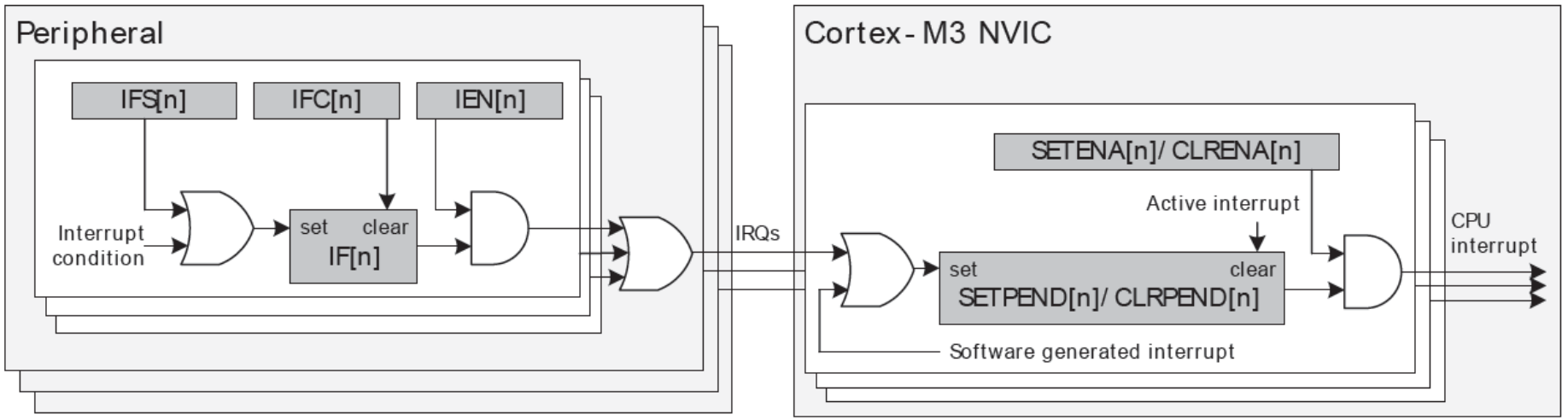


Megszakításkezelő

- Általában vektoros megszakításkezelés
  - A memóriában található egy vektortáblázat. A megszakításrendszer több féle módon működhet, pl.:
    - Megszakítás esetén a program a megszakítás által meghatározott címre ugrik, és onnan folytatódik a program végrehajtása.
      - Itt vagy gyorsan lekezeljük a megszakítást (max. néhány asm utasítás): ritka
      - Vagy elugrunk egy megszakításkezelő függvényre.
    - Megszakítás esetén a vektrotáblában elhelyezett címre ugrik a program (hosszabb kiszolgálás esetén gyorsabb, mert eleve oda ugrunk, ahol a teljes megszakításrutin lefuthat).
  - Előfordulhat vektoros megszakítás esetén is, hogy adott megszakításhoz több esemény is tartozhat
    - Példa: az EFM32 fejlesztői kártyán a GPIO-khoz két megszakítás tartozik: páratlan és páros. A megszakításrutinban kell meghatározni, hogy melyik GPIO volt a forrás.

Példa: megszakítás EFM32 + Cortex-M3

- Két szintű IT:
  - Periféria
    - Interrupt condition: interrupt esemény
    - IFS: Interrupt Flag Set: szoftveresen is lehet IT-t generálni
    - IFC: Interrupt Flag Clear: Interrupt flag törlése
    - IEN: adott interrupt esemény engedélyezése
  - Cortex-M3 mag (NVIC: Nested Vector Interrupt Controller)
    - Az egyes perifériáktól jövő IT esemény egy adott IRQ (Interrupt Request) vonalra képződik le
    - IT esetén PEND bit 1-es lesz, az IT rutinba belépéskor ez törlődik (DE a periféria IT flagje nem!)



Példa: megszakítás EFM32 + Cortex-M3

- Processzormag megszakításstáblája
  - Negatív azonosítójú megszakítások a processzormaghoz tartoznak
  - Pozitív azonosítójú IRQ a processzor perifériáihoz tartoznak
  - A megszakításokhoz tartozik prioritás, ami állítható

Table 2.1. EFM32 Interrupts

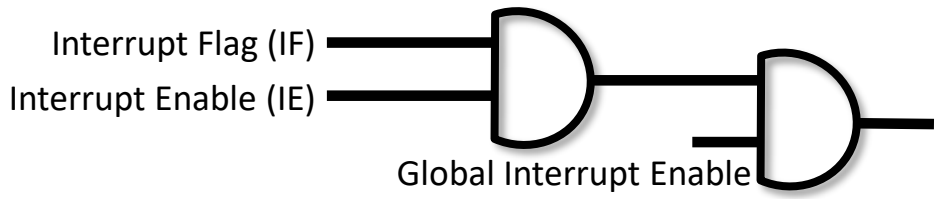
IRQ #	Source
-	Reset
-14	Non-maskable interrupt (NMI)
-13	Hard fault
-12	Memory management fault
-11	Bus fault
-10	Usage fault
-5	SVCall
-2	PendSV
-1	SysTick
0	DMA
1	GPIO_EVEN
2	TIMER0
3	USART0_RX
4	USART0_TX
5	ACMP0/ACMP1
6	ADC0
7	DAC0
8	I2C0
9	GPIO_ODD
10	TIMER1
11	USART1_RX
12	USART1_TX
13	LESENSE
14	LEUART0
15	LETIMER0
16	PCNT0
17	RTC
18	CMU
19	VCMP
20	LCD
21	MSC
22	AES

Példa: AVR (ATmega128)

- Egyszerű vektoros megszakítás
- Adott megszakítás esetén a megszakítástáblában a megfelelő címtől kezdve fut tovább a program.
- A globális IT automatikusan tiltásra kerül.
- ASM kódban RETI paranccsal kell visszatérni (automatikusan visszaállítja globális IT engedélyezést), a C fordítót pedig erre figyelmeztetni kell (lásd később).
- Az IT rutin meghívásakor a hozzá tartozó flag automatikusan törlődik.

Table 23. Reset and Interrupt Vectors

Vector No.	Program Address(2)	Source	Interrupt Definition
1	$0000(1)	RESET	External Pin, Power-on Reset, Brown-out Reset, Watchdog Reset, and JTAG AVR Reset
2	$0002	INT0	External Interrupt Request 0
3	$0004	INT1	External Interrupt Request 1
4	$0006	INT2	External Interrupt Request 2
5	$0008	INT3	External Interrupt Request 3
6	$000A	INT4	External Interrupt Request 4
7	$000C	INT5	External Interrupt Request 5
8	$000E	INT6	External Interrupt Request 6
9	$0010	INT7	External Interrupt Request 7
10	$0012	TIMER2 COMP	Timer/Counter2 Compare Match




```
Address  Labels Code            Comments
$0000          jmp   RESET      ; Reset Handler
$0002          jmp   EXT_INT0   ; IRQ0 Handler
$0004          jmp   EXT_INT1   ; IRQ1 Handler
$0006          jmp   EXT_INT2   ; IRQ2 Handler
$0008          jmp   EXT_INT3   ; IRQ3 Handler
$000A          jmp   EXT_INT4   ; IRQ4 Handler
$000C          jmp   EXT_INT5   ; IRQ5 Handler
$000E          jmp   EXT_INT6   ; IRQ6 Handler
$0010          jmp   EXT_INT7   ; IRQ7 Handler
$0012          jmp   TIM2_COMP  ; Timer2 Compare Handler
```

Megszakítások kezelése

- A megszakítások inicializálása általában a következő módon történik:
  - Periféria engedélyezése
  - Megszakítás-kezelő függvény megadása
  - Adott megszakításhoz tartozó IT flag törlése
    - Esetleg korábbi állapotból bennragadhat egy megszakításkérés, ami problémát okoz, mert az engedélyezés után egyből érvényre jut egy fals megszakítás. A bennragadt IF lehet annak is az eredménye, hogy a perifériát még nem inicializáltuk (pl. lebegő bemeneten véletlenül beütő megszakítás).
  - Adott perifériához tartozó megszakítás engedélyezése
  - Globális megszakítás flag törlése (ha kell)
  - Globális megszakítás engedélyezése

Megszakítások kezelése

- A megszakítások kiszolgálásának javasolt módja:
  - Függvényen belül gyors kiszolgálás
  - Függvényen kívül a nagyobb feladatok lekezelése
  - Ha olyan perifériát használunk, amelyet a program más részében is, akkor a kölcsönös kizárásra figyelni kell. (pl. ha a program egy komponense UART-on küld adatot, és a megszakításrutinban szintén küldünk adatot, akkor a két adatsor összekeveredik).
    - Érdemes az ilyen jellegű feladatokat egy flaggel jelezni, és a főprogramban végrehajtani, ha nem időkritikus.
  - A megszakításkezeléshez szorosan kötődő feladatok (sorend!):
    - Le kell kezelni a perifériát,
    - A megfelelő IT flaget törölni kell (hacsak ez nem automatikus),
    - Ha kell, újra engedélyezni kell a megszakítást (sokszor automatikus).

Megszakítások kezelése

- Mintapélda

Fontos a volatile típus: a Button_IRQ függvény hívását a fordító nem látja, ezért nincsen tudomása róla, hogy a háttérben változhat. A gomb_IT_flag olvasásakor ezért előfordulhat, hogy nem olvassa ki az új értékét.

```
volatile bool gomb_IT_flag = false;
int gombnyomb;

// valahol valahogyan megadjuk, hogy ez a függvény fogja a megszakítást lekezelni: lásd később…
void Button_IRQ(void){  // IT kezelő függvény
    gomb_IT_flag = true; // gyors lekezelés
    clear_button_IF(); // clear button IT flag
}

int main(void){

initButtons(); // gombok inicializálása
clear_button_IF(); // IT flag törlése
Button_IT_enable(); // gombokhoz tartozó IT engedélyezése
clear_global_IF(); // IT flag törlése
global_IT_enable(); // globális IT engedélyezése

while(1){
    if (gomb_IT_flag ){
        gomb_IT_flag = false;
        printLCD(”gombnyomas: %d”, gombnyomb++); // a kiírás lassú, így a főprogramban végezzük
    }
  }
}
```

Megszakításkezelő megadása

- Alapszabály: „Nincs szabály”
  - Sok féle megoldás van
  - Mindig utána kell nézni, hogy melyik fordító/processzor esetén hogyan kell csinálni (RTFM)
- A C nyelv alapból nem támogatja a megszakításkezelést, így fordítónként és eszközönként különféle megoldások keletkeztek
- Habár általános szabály nem létezik, vannak „receptek”, irányelvek.

Megszakításkezelő megadása – EFM32

- A gyártó által szolgáltatott ún. startup kódban definiálva vannak a megszakításvektorhoz tartozó címek
- Minden címhez később egy default IT handlert rendelnek (egy végtelen ciklus)
- A saját kódunkban egy, a vektortáblában található névvel megegyező nevű, paraméter nélküli függvényt kell megadni.
- A startup kódban a függvények weak attribútummal vannak ellátva, ami azt jelenti, hogy ezen függvényeket a kódunkban újra létrehozva nem kapunk hibaüzenetet.
- Ezt követően a megszakítás létrejöttekor a saját függvényünk hívódik meg a vektortáblából (a linker a startup kódban megadott függvénynév helyére a mi függvényünk címét helyettesíti, nem a weak attribútummal ellátott default függvényt).

```
/* Cortex-M Exception Handlers */
{ .topOfStack = &__INITIAL_SP },
{ Reset_Handler                },
{ NMI_Handler                  },
{ HardFault_Handler            },
{ MemManage_Handler            },
```

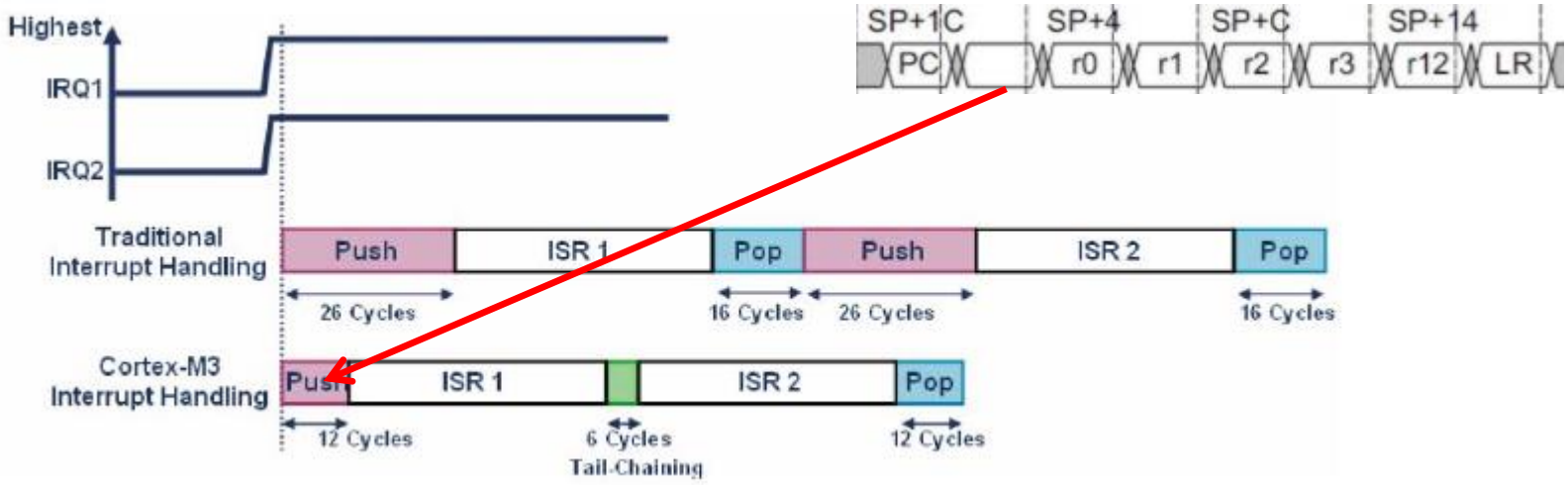
```
/*----------------------------------------------------------------------------
 * Exception / Interrupt Handler
 *----------------------------------------------------------------------------*/
/* Cortex-M Processor Exceptions */
void NMI_Handler(void)        __attribute__ ((weak, alias("Default_Handler")));
void HardFault_Handler(void)  __attribute__ ((weak, alias("Default_Handler")));
void MemManage_Handler(void)  __attribute__ ((weak, alias("Default_Handler")));
void BusFault_Handler(void)   __attribute__ ((weak, alias("Default_Handler")));
```

```
/*----------------------------------------------------------------------------
 * Default Handler for Exceptions / Interrupts
 *----------------------------------------------------------------------------*/
void Default_Handler(void)
{
  while (true) {
  }
}
```

Megszakításkezelő megadása – EFM32

- Egyszerű az IT kezelés:
  - A Cortex – M3 processzorok a megszakítási eseménykor több fontos regisztert is automatikusan elmentenek (R0…R3, R12, PC, Link Reg.) így a megszakításkezelés általában egy egyszerű függvénnyel is megoldható.
  - Interrupt latency (IT létrejötte és kiszolgálásának kezdete közti idő) alacsony
  - Példa (GPIO IRQ kezelő):

```
void GPIO_ODD_IRQHandler(void){ …
    Függvényimplementáció…}
```



Megszakításkezelő megadása – EFM32

- Biztonságos megoldás: meg kell mondani valahogyan a fordítónak, hogy ez egy speciális IT kezelő függvény, mert ekkor biztosan lementi minden olyan regiszter állapotát, amelyet módosított.
  - Főleg akkor fontos, ha nem csak ezen a típusú processzoron szeretnénk a programot futtatni.
- Hogyan mondjuk meg a fordítónak, hogy ez egy megszakításkezelő függvény?
  - Az **__attribute__** speciális csatolóval a függvényeknek és változóknak különböző tulajdonságait állíthatjuk be, amelyeket a fordító értelmez.
  - IT rutin megadása (pl. GPIO IRQ kezelő):

```
void GPIO_ODD_IRQHandler(void) __attribute__ ((interrupt ("IRQ")));
void GPIO_ODD_IRQHandler(void){ …
    Függvény implementáció…}
```

Megszakításkezelő megadása – EFM32

- Milyen kód fordul normál függvény és IRQ attribútummal ellátott függvény esetén?
  - Több regisztert ment le (lásd push parancs), de egy részét redundánsan, ugyanis a processzor ezeket alapból lementi.
  - Ha valaki nem ismeri az adott processzort, akkor biztosabb megoldás (és lehet, hogy a C kód olyan regisztert is fog használni, ami nem kerül lementésre…)

__attribute__ ((interrupt ("IRQ")))

```
13          void GPIO_ODD_IRQHandler(void){
            GPIO_ODD_IRQHandler:
000004d8:     mov     r0,sp
000004da:     bic     r1,r0,#0x7
000004de:     mov     sp,r1
000004e0:     push    {r0,r3,r7,lr}
000004e2:     add     r7,sp,#0x0
14                GPIO_IF_value = GPIO->IF;
000004e4:     ldr     r3,[pc,#0x20] ; 0x504
000004e6:     ldr.w   r3,[r3,#0x114]
000004ea:     ldr     r2,[pc,#0x20] ; 0x508
000004ec:     str     r3,[r2]
15                GPIO_IntClear(1<<PUSH1_PIN); // if commented, IF
000004ee:     mov.w   r0,#0x200
000004f2:     bl      0x00000320
16                GPIO_PinOutToggle(LED1_PORT, LED1_PIN);
000004f6:     movs    r0,#0x4
000004f8:     movs    r1,#0x2
000004fa:     bl      0x00000444
17          }
000004fe:     mov     sp,r7
00000500:     pop.w   {r0,r3,r7,lr}
00000504:     mov     sp,r0
00000506:     bx      lr
00000508:     str     r0,[r0]
0000050a:     ands    r0,r0
0000050c:     lsls    r4,r4,#2
0000050e:     movs    r0,#0x0
```

normál függvény

```
13          void GPIO_ODD_IRQHandler(void){
            GPIO_ODD_IRQHandler:
000004d8:     push    {r7,lr}
000004da:     add     r7,sp,#0x0
14                GPIO_IF_value = GPIO->IF;
000004dc:     ldr     r3,[pc,#0x18] ; 0x4f4
000004de:     ldr.w   r3,[r3,#0x114]
000004e2:     ldr     r2,[pc,#0x18] ; 0x4f8
000004e4:     str     r3,[r2]
15                GPIO_IntClear(1<<PUSH1_PIN); // if commented, IF is
000004e6:     mov.w   r0,#0x200
000004ea:     bl      0x00000320
16                GPIO_PinOutToggle(LED1_PORT, LED1_PIN);
000004ee:     movs    r0,#0x4
000004f0:     movs    r1,#0x2
000004f2:     bl      0x00000444
17          }
000004f6:     pop     {r7,pc}
000004f8:     str     r0,[r0]
000004fa:     ands    r0,r0
000004fc:     lsls    r4,r4,#2
000004fe:     movs    r0,#0x0
```

Megszakításkezelő megadása – AVR

- Példa: ATmega128 (8 bites uC)
- Meg kell adni az IT nevét és a programkódot:

ISR(vektor_neve)

Példa:

```
ISR(USART1_RX_vect) {
    UART1_read_data();
    …  // mást is csinálhatunk még
    crear_IF();
}
```

- Lehetnek egyéb csatolók, pl. „ISR_NOBLOCK”: az IT rutinban már engedélyezi a fordító a többi megszakítást. Példa: ISR(USART1_RX_vect, ISR_NOBLOCK){…}

Megszakításkezelő megadása – AVR

- Hogyan működik?

```
#  define ISR(vector, ...)            \
   void vector (void) __attribute__ ((signal,__INTR_ATTRS)) __VA_ARGS__; \
   void vector (void)
```

Tehát például a „ ISR(USART1_RX_vect){…}” parancsból a következő fordul a makró kifejtése után:

```
void __vector_30 __attribute__ ((signal));  //függvény deklarációja attribútummal együtt
void __vector_30{     //függvény implementáció
     kód
}
```

Magyarázat (valahol egy másik helyen a következő definíciók láthatók):

```
#define USART1_RX_vect            _VECTOR(30)
…
#define _VECTOR(N) __vector_ ## N
```

A __vector_30 cím pedig a crtm128.o startup fájlban található, ezt definiáljuk felül.

Megszakításkezelő megadása – ADSP21364

- Példa: ADSP21364: 32 bites, lebegőpontos DSP
- Callback függvény: adott esemény esetén meghívott függvények (lásd pl. Java)
- A gyártó létrehozott egy IT diszpécser függvényt. A függvény minden megszakítás esetén lefut, lementi a processzor állapotát, és meghívja az általunk az adott megszakításhoz rendelt függvényt (callback függvény jellegű).
- A megszakításkezelő függvény egy „sima” függvény, a kontextusváltást a diszpécser végzi.
- Létezik többféle opció. A következő függvények segítségével lehet regisztrálni a megszakításkezelő függvényeket:
  - interruptcb(): a megszakítás során minden speciális DSP HW állapotát lementi (pl. cirkuláris buffer, loop counter, másodlagos regiszterek) kb. 200 órajelciklus
  - interrupt(): a megszakítás során a legtöbb speciális DSP HW állapotát lementi kb. 150 órajelciklus
  - interruptf(): gyors kiszolgálás, nem menti le minden speciális HW állapotát
  - Stb…: lényeg, hogy különböző szinten menti le a processzor aktuális állapotát. Minél több részletet ment le a processzorról, annál lassabb a kontextusváltás, de annál megbízhatóbb a futás. Gyors kontextusváltás esetén figyelni kell, hogy mit csinálhatunk a függvényben.

Megszakításkezelő megadása – ADSP21364

- Példa: UART megszakításának kezelése
  - Ki kell keresni a fordító dokumentációjából, hogy az adott processzor esetén hogyan nevezték el az UART-hoz tartozó perifériát. Esetünkben ennek a neve SIG_SP4 (SP: serial port)
  - regisztráljuk a függvényünket, esetünkben pl. UARTrec
  - Implementáljuk a függvényt

```
interrupt (SIG_SP4 , UARTrec); // SIG_SP4 típus és kezelő függvény

// megadjuk a kezelő függvényt:
void UARTrec(void){
   // valami megszakításkezelés
}
```

Megszakításkezelő megadása-összefoglalás

- **Általános feladatok:** periféria felkonfigurálása, IT flagek törlése, IT engedélyezése (periféria és globális), megszakításrutinban periféria lekezelése (sokszor az IF „kézi” törlése).
- **EFM32-Cortex M3:** A startup fájlbol kikeresni a megszakításkezelő nevét, és ezzel a névvel egy saját függvényt megadni (esetleg interrupt attribútummal). Példa:

```
void GPIO_ODD_IRQHandler(void) __attribute__ ((interrupt ("IRQ")));
void GPIO_ODD_IRQHandler(void){
   Függvény implementációja…}
```

- **AVR 8 bites mikrokontroller:** Adott IT-hez tartozó név kikeresése a dokumentációkból, és az ISR() makró felhasználásával a függvényt megadjuk. Példa:

```
ISR(USART1_RX_vect) {
   Függvény implementációja…}
```

Megszakításkezelő megadása-összefoglalás

- Mindig el kell olvasni az adott processzor és fordító dokumentációit:
  - Kell-e jelölni, hogy ez egy speciális IT kezelő függvény? (pl. #pragma, __attribute__)
  - Törli-e a processzor automatikusan az IT flaget? Ha nem, akkor tegyük meg, de egyébként is érdemes.
  - Belépéskor törli-e a processzor megszakítás-engedélyezést?
  - Van-e többszintű megszakítás?
  - Kell-e kilépéskor engedélyezni a megszakítást?
- A perifériát le kell kezelni! (pl. UART kiolvas, IF töröl)
- Használok-e speciális perifériát, változót? Ha igen, akkor biztosítom-e a kölcsönös kizárást?
- Ne legyen túl hosszú a megszakításrutin.
- A megszakításkezelő függvények általában paraméter nélküli függvények. Ha valamilyen paraméterre van szükség, azt magunknak kell megoldani.
- Tartozik-e több forrás ugyanahhoz a megszakításvonalhoz? (Ha igen, le kell kérdezni.)
- A megszakításrutinokban írt változó mindig legyen **volatile** típusú.
  - A voltaile változókat a fordító úgy kezeli, hogy bármikor változhatnak, így minden egyes hivatkozás során újra kiolvassa az értékét, még ha látszólag az nem is változhat.

Megosztott változók



Mesterséges Intelligencia és Rendszertervezés Tanszék

© BME-MIT

Megosztott változók

- Probléma: egy adott változóhoz, vagy bármilyen adathoz több folyamat **időben átlapoltan** fér hozzá. Tipikusan például:
  - Megszakítás és megszakítás között
  - Megszakítás és főprogram (valamelyik task) között
  - Preemptív futtató rendszerek esetén
- Akkor léphet fel probléma, ha nem atomikusan lehet írni/olvasni az adato(ka)t vagy bármilyen, összetartozó információkat tároló adatstruktúrát.
  - Atomikus művelet: más folyamat nem szakíthatja meg a program futását az adott művelet végzése közben.
  - Magas szintű nyelvben látszólag egységesnek tűnő utasítás is lehet nem atomikus. Például egy 32 bites szám inkrementálása egy 32 bites architektúrán *lehet* atomikus, míg egy 8 bites architektúrán biztosan nem az, mivel a processzor csak több assembly utasításban tudja csak elvégezni a műveletet.

Megosztott változó: példa

- Az `adattomb` nevű változót használja a főprogram és a timer megszakításrutin.

```
// adatstruktura deklaracioja
struct Tipus{
      uint32_t adat1;
      uint32_t adat2;
} adattomb;
// timer IT kezeles
void LETIMER0_IRQHandler(void){
      // adattomb irasa
      adattomb.adat1 = 0xFF;
      adattomb.adat2 = 0xFF;
      LETIMER0->IFC = 0xFF;  // IT flag torlese
}
void main(void){
  // vegtelen ciklus
while (1) {
      // adattomb irasa
      adattomb.adat1 = 0x00;
      adattomb.adat2 = 0x00;
// elvileg nem lephetunk be ide, hiszen vagy 0x00 vagy 0xFF mindket valtozo tartalma
// feltetel: ha nem igaz, hogy az adat1 és adat2 is egyszerre 0xFF vagy 0x00 értékű
      if (!( (adattomb.adat1==0xFF && adattomb.adat2==0xFF) ||
             (adattomb.adat1==0x00 && adattomb.adat2==0x00)) ){
            hiba = true;
            BSP_LedToggle(1);
      } } }
```

Megosztott változó: példa

- Probléma: az adattomb struktúra adat1 és adat2 mezőjét nem atomikusan írjuk:
  - A megszakításrutin akár abban a pillanatban is lefuthat, amikor az `adattomb.adat1 = 0x00;` utasítás lefutott a főprogramban, de az adat2 értékadás még nem. Ekkor a következő történik:
    - `adattomb.adat1 = 0x00;  // főprogram`
    - `Megszakításrutin elindul`
    - `adattomb.adat1 = 0xFF;  // megszakítás`
    - `adattomb.adat2 = 0xFF;  // megszakítás`
    - `Megszakításrutin vége`
    - `adattomb.adat2 = 0x00;  // főprogram`
  - Végeredmény: adat1=0xFF az adat2=0x00 lesz, tehát inkonzisztens adatok lesznek.

Megosztott változó: példa

- Megoldás: az adattomb struktúra adat1 és adat2 mezőjét atomikusan kell írnunk.
  - A megszakításrutinban ez egyértelműen teljesül, mert nincsen másik megszakításrutin. Ha lenne nagyobb prioritású megszakításrutin, akkor ott is kellene biztosítani az atomikus írást.
  - A főprogramban midenképpen kell biztosítani az atomikus írást. Hogyan?
    - Egyszerű megoldás: tiltjuk a megszakítást az írás kezdetekor és engedélyezzük az írás végekor.
    - A fordítók/processzorok függvénykönyvtárai sokszor biztosítanak lehetőséget atomikus szakaszok kijelölésére. Lényegében ez is az interruptot tiltja/engedélyezi, de vizsgálja, hogy a kritikus szakasz elején volt-e engedélyezve a megszakítás.

Megosztott változó: példa

- Egészítsük ki a főprogramot, és a változót írjuk atomikusan. Az adott környezetben (Simplicity Studio – Cortex-M3) a CORE_CRITICAL_SECTION(…) makróval határolt rész egy atomikus művelet lesz.

```
while (1) {
       // kritikus szakasz: atomikus művelet az írás
      CORE_CRITICAL_SECTION(
            // adattomb irasa
            adattomb.adat1 = 0x00;
            adattomb.adat2 = 0x00;
      )

// elvileg nem lephetunk be ide, hiszen vagy 0x00 vagy 0xFF mindket valtozo tartalma
// feltetel: ha nem igaz, hogy az adat1 és adat2 is egyszerre 0xFF vagy 0x00 értékű
      if (!( (adattomb.adat1==0xFF && adattomb.adat2==0xFF) ||
             (adattomb.adat1==0x00 && adattomb.adat2==0x00)) ){
            hiba = true;
            BSP_LedToggle(1);
      } } }
```

Megosztott változó: példa

- Egészítsük ki a főprogramot, és a változót írjuk atomikusan. Az adott környezetben (Simplicity Studio – Cortex-M3) a CORE_CRITICAL_SECTION(…) makróval határolt rész egy atomikus művelet lesz.

```
while (1) {
       // kritikus szakasz: atomikus művelet az írás
      CORE_CRITICAL_SECTION(
            // adattomb irasa
            adattomb.adat1 = 0x00;
            adattomb.adat2 = 0x00;
      )

// elvileg nem lephetunk ide, hiszen vagy 0x00 vagy 0xFF mindket valtozo tartalma
// feltetel: ha nem igaz, hogy adat1 és adat2 is egyszerre 0xFF vagy 0x00 értékű
      if (!( (adattomb.adat1==0xFF && adattomb.adat2==0xFF) ||
             (adattomb.adat1==0x00 && adattomb.adat2==0x00)) ){
            hiba = true;
            BSP_LedToggle(1);
      } } }
```

Ez a rész még mindig nem atomikus!

Megosztott változó: példa

- Még mindig nem jó valami …
- Befutunk abba a részbe, ahova nem lenne szabad (egy breakpoint-ra fut rá):

```
94          if (!( (adattomb.adat1==0xFF && adattomb.adat2==0xFF) ||
95                 (adattomb.adat1==0x00 && adattomb.adat2==0x00)) ){
96              hiba = true;
97              BSP_LedToggle(1);
98          }
```

- Ráadásul mind az adat1 és az adat2 változó értéke is 0xff, tehát elvileg a feltétel nem is teljesül (akkor **nem** fut be az **if** feltételbe, ha vagy mind a két változó 0xff vagy mindkét változó 0x00) ..

Name	Type	Value	Location
adattomb	struct Tipus	0x200000a4	0x200000a4
(x)= adat1	uint32_t	0xff	0x200000a4
(x)= adat2	uint32_t	0xff	0x200000a8

- Mi lehet a probléma?

Megosztott változó: példa

- Van még egy „rejtett” olvasás a programban:
  - Az **if** (…) feltétel kiértékelésekor ki kell olvasni a változókból az értéket. A következő történhet:
    - Elkezdjük kiértékelni az

      (adattomb.adat1==0x00 && adattomb.adat2==0x00)

      feltételt.
    - Az (adattomb.adat1==0x00) részfeltétel kiértékelésekor még az adat1 és adat2 változó értéke 0x00, tehát ez a részfeltétel igaz.
    - Ezután beüt egy megszakítás, és „hirtelen” mindkét változó 0xFF értékű lesz. Tehát a (adattomb.adat2==0x00) részfeltétel „hirtelen” hamis lesz.
    - A részfeltétel tehát hamis, így a teljes **if** feltétel végül igaz lesz (az elején van egy invertálás).
  - A feltétel kiértékelése tehát szintén kritikus szakaszba teendő, hogy atomi művelet legyen.
    - Megjegyzés: ebben a struktúrában nem lehet a feltétel kiértékelését különválasztani

Megosztott változó: példa

- A helyes megoldás, hogy az if feltételt is egy kritikus szakaszba tesszük:

```
while (1) {
       // kritikus szakasz: atomikus művelet az írás
      CORE_CRITICAL_SECTION(
            // adattomb irasa
            adattomb.adat1 = 0x00;
            adattomb.adat2 = 0x00;
      )

// elvileg nem lephetunk be ide, hiszen vagy 0x00 vagy 0xFF mindket valtozo tartalma
// feltetel: ha nem igaz, hogy az adat1 és adat2 is egyszerre 0xFF vagy 0x00 értékű
CORE_CRITICAL_SECTION(
      if (!( (adattomb.adat1==0xFF && adattomb.adat2==0xFF) ||
                              (adattomb.adat1==0x00 && adattomb.adat2==0x00)) ){
            hiba = true;
            BSP_LedToggle(1);
      }
      )
} }
```

Megosztott változó: példa

- A mi konkrét példánkban a megszakításrutinban nem kell atomikus írás, mert ezt más folyamat már nem szakíthatja meg, így lényegében atomikusnak tekinthető.
- Most már helyesen működik a program, de lehet tovább finomítani.
- Probléma:
  - A kritikus szakasz nem csak a ténylegesen atomikusan végrehajtandó feltételkiértékelést tartalmazza, hanem a feltétel kiértékelése után feltételesen végrehajtandó parancsokat is, ami túl hosszúra nyújthatja a kritikus szakasz hosszát.
  - Eredmény: túl sokáig lehet letiltva a megszakítás.
  - Nem lehet csak az if feltétel-kiértékelést kritikus szakaszba tenni (tiltva marad a megszakítás, ha a feltétel utáni kód nem fut le):

```
CORE_CRITICAL_SECTION(
    if (...)
) {  ... programkód ...}
```

- Megoldások:
  - Egy kritikus szakaszon belül lokális másolatot készítünk a változókról, és ezekkel dolgozunk tovább.
  - A ténylegesen atomikusan kiértékelendő részeket külön választjuk és csak ezeket hajtjuk végre a kritikus szakaszon belül (a példában ez a feltétel kiértékelése)

Megosztott változó: példa

- Első megoldás: lokális másolat a változókról, és ezekkel dolgozunk tovább

```
// kritikus szakasz: lokális másolat készítése
CORE_CRITICAL_SECTION(
        adattomb_loc.adat1 = adattomb.adat1;
        adattomb_loc.adat2 = adattomb.adat2;
)
        // a lokális adatokkal dolgozunk tovább
        if (!( (adattomb_loc.adat1==0xFF && adattomb_loc.adat2==0xFF) ||
                        (adattomb_loc.adat1==0x00 && adattomb_loc.adat2==0x00)) ){
                hiba = true;
                BSP_LedToggle(1);
        }
```

Megosztott változó: példa

- Második megoldás: feltétel kiértékelése atomikusan:

```
// kritikus szakasz: feltétel kiértékelése
CORE_CRITICAL_SECTION(
   feltetel = ! ( (adattomb.adat1==0xFF && adattomb.adat2==0xFF) ||
            (adattomb.adat1==0x00 && adattomb.adat2==0x00));
)
if (feltetel){
       hiba = true;
       BSP_LedToggle(1);
}
```

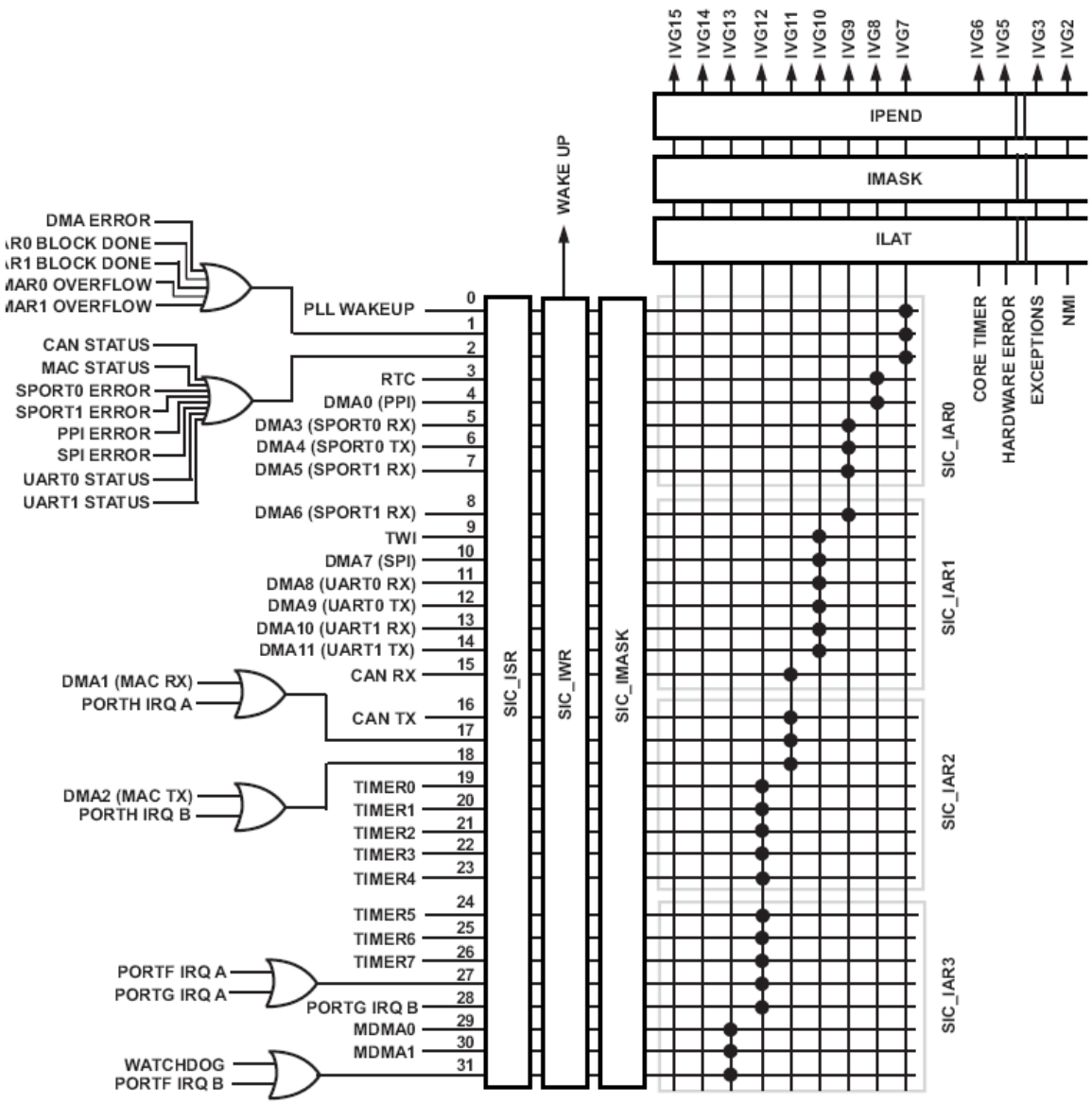
Megosztott változók: összefoglalás

- Ha egy adatot vagy adattömböt több helyről írunk és olvasunk, akkor adatinkozisztencia alakulhat ki, amennyiben az írás és olvasás nem atomikus művelet a teljes adatra/adatsorra nézve.
  - Ez processzorfüggő is lehet, hogy milyen adatot tudunk atomikusan kezelni (pl. 8 vagy 32 bites mikrokontroller)
- Minden olyan helyen, ahol hozzáférünk a védett adatokhoz, azt csak atomikusan, egy kritikus szakaszban tehetjük meg. Atomikus szakaszok kialakítása például:
  - Tiltjuk/engedélyezzük a megszakításokat.
  - Kihasználjuk a processzorhoz kapott függvénykönyvtárakat, és valamilyen kritikus szakaszt jelölő sémát használunk.
  - Beágyazott operációs rendszerekben erre vannak beépített szinkronizációs primitívek, például szemaforok, üzenetek...
- Ahhoz, hogy ne blokkoljuk sokáig a program futását a megosztott változók felhasználásakor:
  - Csak a konkrét adatokat tartalmazó kódrészleteket futtatjuk kritikus szakaszban.
  - Lokális másolatokat készítünk a változókról egy kritikus szakaszban, és ezekkel a másolatokkal dolgozunk tovább.

Kiegészítő olvasmány

Példa: BF537 (DSP)

- Analog Devices gyártmányú BF537 típusú DSP megszakítási hierarchiája
- SIC: System Interrupt Controller
- Bal oldalról jönnek az egyes perifériák megszakításvonalai
- SIC_ISR: az elsődleges megszakítások tárolása (IT flag)
- SIC_IWR: processzor felébresztéséhez engedélyezés
- SIC_MASK: elsődleges források engedélyezése
- Kapcsolómátrix: melyik vonalat melyik bemeneti IT vektorra képezzük le
- ILAT: IT latch regiszter
- IMASK: IT maszk regiszter
- IPEND: IT függőben (pending)
- IVGnn: megszakítási vonalak



Példa: BF537 (DSP)

- IT esetén az SIC_ISR (Interrupt Status Regiszter) megfelelő bitje 1-es
- Az SIC_ISR regiszter nem törölhető szoftveresen, ha az adott perifériát kiszolgáltuk, automatikusan törlődik (a periféria törli)
- Egy kimeneti IT vonalhoz (IVG) akár több bemeneti IT forrás is csatlakozhat, a processzor feladata lekérdezni a forrást
  - Ha a fejlesztés során a megszakítások száma nem túl sok, akkor minden forrás külön megszakításvektorhoz rendelhetünk, így a forrás egyértelmű
- Az adott IT vonalhoz tartozó függvényből RTI utasítással visszatérve az adott IT vonal újra engedélyezve lesz (IT során tiltjuk, hogy ne generáljon további megszakítást), és az IPEND regiszter is törlésre kerül.
- Van néhány speciális IT: NMI (nem maszkolhato IT), reset, HW error, core timer

Megszakításkezelő megadása – BF537

- A gyártó által kiadott környezet biztosít olyan függvényeket, amelyek segítségével regisztrálhatjuk a megszakításkezelő függvényeket:

```
register_handler_ex(interrupt_kind kind, ex_handler_fn fn, int enable);
```

Például:

```
register_handler_ex(ik_timer, timer_handler, EX_INT_ENABLE);
```

A függvényt pedig a következő módon kell megadni, például:

```
EX_INTERRUPT_HANDLER(timer_handler){
    Timer_IT_Number ++;
}
```

Az időzítő megszakításkezelőjeként a timer_handler nevű függvényt adjuk meg, és engedélyezzük is a megszakítást.

- Hogyan működik?

Megszakításkezelő megadása – BF537

- Hogyan tesszük be a vektortáblába a függvényünket? (crt\reghdlr.h és sys\exception.h)
  - A vektortábla kezdőcímére ráállítunk egy függvénypointert
  - A függvénypointer egy tömböt definiál, aminek a interrupt_kind elemét átírjuk az általunk adott függvény címére, innentől kezdve ezt hívja meg a processzor

```
register_handler_ex(interrupt_kind kind, ex_handler_fn fn, int enable);

typedef void (*ex_handler_fn)(); // függvénypointer típus megadása
#define EX_EVENT_VECTOR_TABLE  0xFFE02000 // az IT vektortábla kezdőcíme
// a vektortábla kezdőcímére ráállítunk egy függvénytömböt
ex_handler_fn *evt = (volatile ex_handler_fn *)EX_EVENT_VECTOR_TABLE;
// a vektortábla adott elemét (amilyen típusú az IT) átírjuk a saját függvény pointerére
evt[kind] = fn;
```

Megszakításkezelő megadása – BF537

- Hogyan működik az IT kezelő függvény deklarálása? Azt már „tudja” a processzor, hogy melyik függvényt kell meghívni (annak a függvénynek a címét helyeztük a vektortáblába), de honnan tudja a fordító, hogy ezt a függvényt máshogyan kell fordítania? Nézzük a következő makrókat (\sys\exception.h)

```
#define EX_INTERRUPT_HANDLER(NAME)  EX_HANDLER(interrupt,NAME)
#define EX_HANDLER(KIND,NAME) \
_Pragma(#KIND) \
void NAME ()
```

Tehát a

```
EX_INTERRUPT_HANDLER(timer_handler){
    Timer_IT_Number ++;
}
```

Függvénydefiníció kifejtése a következő lesz a makrók behelyettesítése után

```
_Pragma(interrupt)  // innen tudja a fordító, hogy ez egy IT kezelő függvény
void timer_handler(){ // itt pedig megadjuk a függvényt normál módon
    Timer_IT_Number ++;
}
```

Megszakításkezelő megadása – BF537

- Meg lehet adni megszakítható függvényt is, ilyenkor a következő makrót kell használni:

```
EX_REENTRANT_HANDLER(NAME)
```

- A makró kifejtése:

```
#define EX_REENTRANT_HANDLER(NAME) \
_Pragma("interrupt_reentrant") \
EX_HANDLER(interrupt,NAME)
```

- Tehát a _Pragma("interrupt_reentrant") fordítási direktívával jelezzük a fordítónak, hogy ez a függvény megszakítható megszakításkezelő. Olyan kódot generál, amely már a függvény elején újra engedélyezi a megszakításokat.

Megszakításkezelő megadása – BF537

- Összefoglalva:
  - Létezik olyan könyvtári függvény, amely a megszakítási vektortáblában a megszakítási rutin kezdőcímét átírja az álalunk megadott függvényre.
  - Egy gyártó által megadott makró segítségével olyan függvényt hozunk létre, amelyről a fordító is tudni fogja, hogy ez egy megszakításkezelő függvény (Pragma fordítási kulcsszó segítségével)

Megszakításkezelő megadása-összefoglalás

- **Általános feladatok:** periféria felkonfigurálása, IT flagek törlése, IT engedélyezése (periféria és globális), megszakításrutinban periféria lekezelése (sokszor az IF „kézi” törlése).
- **EFM32-Cortex M3:** A startup fájlbol kikeresni a megszakításkezelő nevét, és ezzel a névvel egy saját függvényt megadni (esetleg interrupt attribútummal). Példa:

```
void GPIO_ODD_IRQHandler(void) __attribute__ ((interrupt ("IRQ")));
void GPIO_ODD_IRQHandler(void){
   Függvény implementációja…}
```

- **AVR 8 bites mikrokontroller:** Adott IT-hez tartozó név kikeresése a dokumentációkból, és az ISR() makró felhasználásával a függvényt megadjuk. Példa:

```
ISR(USART1_RX_vect) {
   Függvény implementációja…}
```

- **ADSP BF357 (DSP):** Regisztráljuk a megszakításfüggvényt a vektortáblában, és implementáljuk az EX_INTERRUPT_HANDLER makró segítségével:

```
register_handler_ex(ik_timer, timer_handler, EX_INT_ENABLE);
EX_INTERRUPT_HANDLER(timer_handler){
    Függvény implementációja…}
```

- **ADSP 21364 (DSP):** Regisztráljuk a megszakításfüggvényt a megszakítás dispatchernél:

```
interruptcb(SIG_IRQ1,IRQ1_handler); // IRQ1 típus és kezelő függvény
    void IRQ1_handler(int x){
       Függvény implementációja…}
```